RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CD23321-Python Programming for Design

Python Assignment Report

Name : PRAJIT R

Year / Branch / Section : Computer Science and Design, 2nd Year

Register No. : 2116251701040

Semester : 3rd Semester

Academic Year : 2026 - 2027

SPACE WARS GAME DEVELOPMENT USING PYTHON

AIM:-

To design and develop an interactive 2D Space Wars shooter game using Python Tkinter that allows the player to control a spaceship using the keyboard, shoot bullets at waves of enemy ships, dodge enemy fire, earn points, and progress through increasingly faster waves. The game includes a graphical interface, score, lives and wave tracking, game-over detection, and a restart option.

PROCEDURE:-

1. Start the program and import the required Python libraries: tkinter and random.

2. Define the game constants: window width, window height, and the frame delay (16 ms, about 60 frames per second).

3. Create a Game class that holds only the game logic (player position, bullets, enemies, score, lives, wave number) so that it is independent of the graphical interface.

4. Create the main game window using Tkinter, set the title as "Space Wars", and add a black Canvas widget as the game board.

5. Draw the player ship as a triangle at the bottom of the board and the enemies as red rectangles arranged in rows and columns at the top.

6. Implement keyboard controls: Left/Right arrow keys or A/D to move the ship, Space to shoot, and R to restart after the game is over. Use the KeyPress and KeyRelease events to track which keys are held down.

7. Run the game loop using Tkinter's after() method. In each frame, update the game state, redraw the canvas, and schedule the next frame.

8. Move the player bullets upward and the enemy bullets downward. Move the enemy formation sideways, and make it reverse direction and step downward when it reaches a screen edge.

9. Let a random enemy fire a bullet using the random module.

10. Implement collision detection: a player bullet that hits an enemy removes both and adds 10 points; an enemy bullet that hits the player removes one life.

11. End the game when all lives are lost or when the enemies reach the player's row, and display the GAME OVER message.

12. When all enemies in a wave are destroyed, start the next wave with a higher enemy speed.

13. Execute the game and verify the ship movement, shooting, scoring, collision detection, wave progression, and restart functionality.

PROGRAM CODE:-

```
import tkinter as tk

import random

WIDTH, HEIGHT = 480, 640

FRAME_MS = 16  # ~60 FPS


class Game:

    """Game logic only: no Tkinter in here, so an AI can use it later."""


    def __init__(self):

        self.reset()


    def reset(self):

        self.player_x = WIDTH / 2

        self.player_y = HEIGHT - 50

        self.bullets = []          # player bullets: [x, y]

        self.enemy_bullets = []    # enemy bullets: [x, y]

        self.cooldown = 0

        self.score = 0

        self.lives = 3

        self.wave = 1

        self.game_over = False

        self.spawn_wave()
```

```
def spawn_wave(self):
    self.enemies = [[60 + c * 50, 60 + r * 40] for r in range(4) for c in range(8)]
    self.enemy_dir = 1
    self.enemy_speed = 1.0 + (self.wave - 1) * 0.4

def step(self, keys):
    if self.game_over:
        return

    # Player movement and shooting
    if "left" in keys or "a" in keys:
        self.player_x -= 5
    if "right" in keys or "d" in keys:
        self.player_x += 5
    self.player_x = max(20, min(WIDTH - 20, self.player_x))

    self.cooldown -= 1
    if "space" in keys and self.cooldown <= 0:
        self.bullets.append([self.player_x, self.player_y - 12])
        self.cooldown = 15

    # Move bullets
    for b in self.bullets:
        b[1] -= 8
    for b in self.enemy_bullets:
        b[1] += 5
    self.bullets = [b for b in self.bullets if b[1] > 0]
    self.enemy_bullets = [b for b in self.enemy_bullets if b[1] < HEIGHT]
```

```
# Move enemies; reverse and drop down at the screen edges
if self.enemies:
    xs = [e[0] for e in self.enemies]
    s = self.enemy_speed
    if (self.enemy_dir > 0 and max(xs) + 15 + s > WIDTH) or \
       (self.enemy_dir < 0 and min(xs) - 15 - s < 0):
        self.enemy_dir *= -1
        for e in self.enemies:
            e[1] += 15
    else:
        for e in self.enemies:
            e[0] += self.enemy_dir * s

# Enemies shoot randomly
if self.enemies and random.random() < 0.03:
    shooter = random.choice(self.enemies)
    self.enemy_bullets.append([shooter[0], shooter[1] + 12])

# Player bullets hit enemies
for b in self.bullets[:]:
    for e in self.enemies:
        if abs(b[0] - e[0]) < 17 and abs(b[1] - e[1]) < 16:
            self.enemies.remove(e)
            self.bullets.remove(b)
            self.score += 10
            break

# Enemy bullets hit player
```

```
        for b in self.enemy_bullets[:]:
            if abs(b[0] - self.player_x) < 20 and abs(b[1] - self.player_y) < 14:
                self.enemy_bullets.remove(b)
                self.lives -= 1
                if self.lives <= 0:
                    self.game_over = True

        # Enemies reached the player
        if any(e[1] + 10 >= self.player_y - 10 for e in self.enemies):
            self.game_over = True

        # Wave cleared: next wave is faster
        if not self.enemies and not self.game_over:
            self.wave += 1
            self.spawn_wave()



class App:
    """Tkinter window: drawing and keyboard input only."""

    def __init__(self, root):
        self.root = root
        root.title("Space Wars")
        root.resizable(False, False)
        self.canvas = tk.Canvas(root, width=WIDTH, height=HEIGHT, bg="black")
        self.canvas.pack()

        self.game = Game()
        self.keys = set()
```

```
    root.bind("<KeyPress>", self.on_key_down)
    root.bind("<KeyRelease>", self.on_key_up)

    self.loop()

def on_key_down(self, event):
    key = event.keysym.lower()
    self.keys.add(key)
    if key == "r" and self.game.game_over:
        self.game.reset()

def on_key_up(self, event):
    self.keys.discard(event.keysym.lower())

def loop(self):
    self.game.step(self.keys)
    self.draw()
    self.root.after(FRAME_MS, self.loop)

def draw(self):
    c = self.canvas
    g = self.game
    c.delete("all")

    # Player ship (triangle)
    x, y = g.player_x, g.player_y
    c.create_polygon(x, y - 12, x - 20, y + 10, x + 20, y + 10, fill="cyan")

    # Enemies
```

```
        for ex, ey in g.enemies:
            c.create_rectangle(ex - 15, ey - 10, ex + 15, ey + 10, fill="red")

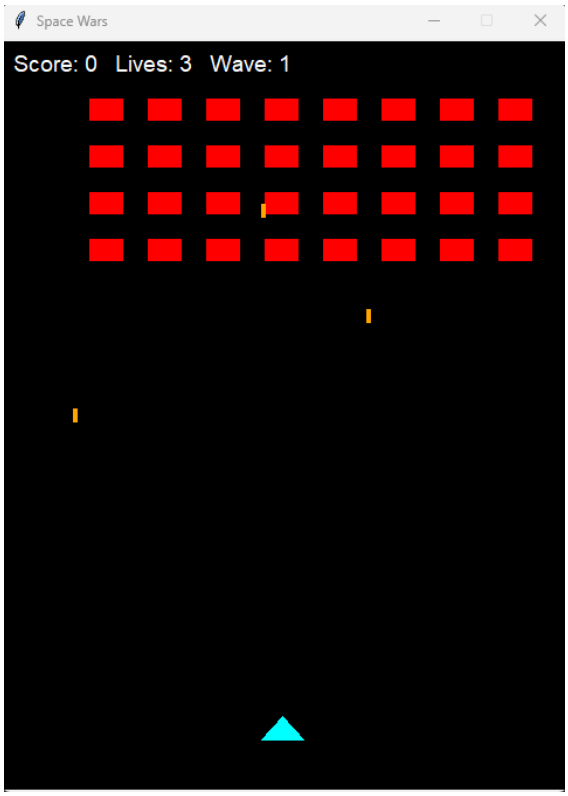
        # Bullets
        for bx, by in g.bullets:
            c.create_rectangle(bx - 2, by - 6, bx + 2, by + 6, fill="yellow", outline="")
        for bx, by in g.enemy_bullets:
            c.create_rectangle(bx - 2, by - 6, bx + 2, by + 6, fill="orange", outline="")

        # HUD
        c.create_text(10, 10, anchor="nw", fill="white", font=("Arial", 14),
                      text=f"Score: {g.score}   Lives: {g.lives}   Wave: {g.wave}")

        if g.game_over:
            c.create_text(WIDTH / 2, HEIGHT / 2 - 20, fill="white",
                          font=("Arial", 32, "bold"), text="GAME OVER")
            c.create_text(WIDTH / 2, HEIGHT / 2 + 25, fill="white",
                          font=("Arial", 16), text="Press R to restart")


if __name__ == "__main__":
    root = tk.Tk()
    App(root)
    root.mainloop()
```

OUTPUT:-


Space Wars
Score: 0 Lives: 3 Wave: 1


Space Wars
Score: 0 Lives: 0 Wave: 1
GAME OVER
Press R to restart

RESULT:-

Thus, an interactive 2D Space Wars game was developed using Python Tkinter with graphical game design, keyboard-based ship movement and shooting, enemy formations with random enemy fire, automatic score calculation, lives and wave tracking, collision detection, game-over handling, and restart functionality. The program demonstrates event-driven programming, GUI design, animation using a game loop, conditional statements, object-oriented programming, and the separation of game logic from the user interface